SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA1402 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Code of Conduct:

I _Monika.R(192324281)_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Monika.R

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	**/ 25**	**/ 20**	**/ 20**	**/ 10**	**/ 100**

SET:2

QUESTIONS

1. Industrial Robot Automation System

A manufacturing company is developing an industrial robot control system where robotic arms perform assembly operations automatically. The compiler used in the embedded software processes declarations, assignment statements, and Boolean expressions to coordinate robotic movement and safety checks. During production, delays occurred due to inefficient handling of procedure calls and intermediate code generation. The company plans to redesign the compiler to improve execution speed and synchronization between robotic units.

Tasks:

(i) Design intermediate code for robotic movement and safety validation logic. *(K3)*

(ii) Explain how procedure calls improve modularity in industrial automation systems. *(K4)*

(iii) Analyze the role of compiler optimization in improving manufacturing efficiency. *(K5)*

2. Cybersecurity Intrusion Detection System

A cybersecurity firm is developing an intrusion detection system that continuously monitors network traffic for suspicious activities. The compiler processes Boolean expressions, assignment statements, and case statements to identify different categories of cyberattacks. During real-time monitoring, inefficient intermediate code generation increased system response time and delayed threat detection. The development team intends to enhance compiler efficiency for faster and more reliable security analysis.

Tasks:

(i) Generate intermediate representation for network threat detection logic. *(K3)*

(ii) Discuss how case statements can efficiently classify multiple cyberattack scenarios. *(K4)*

(iii) Evaluate the importance of compiler optimization in real-time cybersecurity applications. *(K5)*

3. Digital Payment Gateway System

A fintech company is building a digital payment gateway to process online transactions securely and efficiently. The compiler used in the transaction engine must evaluate Boolean conditions, assignment statements, and procedure calls during payment verification and fraud detection. During peak transaction periods, improper backpatching caused delays in payment

confirmation and rollback operations. The organization plans to redesign the compiler to improve transaction reliability and scalability.

Tasks:

(i) Construct intermediate code for payment verification and fraud detection logic. *(K3)*

(ii) Explain how backpatching supports conditional transaction processing. *(K4)*

(iii) Analyze the role of compiler design in ensuring secure and scalable fintech applications. *(K5)*

4. Smart Agriculture Monitoring System

An agritech company is developing a smart agriculture platform that monitors soil moisture, temperature, and irrigation systems automatically. The compiler in the embedded controller processes declarations, assignment statements, and Boolean expressions to activate irrigation based on environmental conditions. Inefficient handling of intermediate code caused delays in irrigation control during critical farming conditions. The software team aims to improve compiler performance for real-time agricultural automation.

Tasks:

(i) Develop intermediate code for irrigation control and environmental monitoring. *(K3)*

(ii) Explain the role of Boolean expressions in automated farming systems. *(K4)*

(iii) Evaluate how compiler optimization enhances efficiency in smart agriculture applications. *(K5)*

5. Cloud-Based Video Streaming Platform

A multimedia company is developing a cloud-based video streaming platform that dynamically adjusts video quality based on network conditions. The compiler used in the streaming engine processes case statements, assignment statements, and procedure calls for resource allocation and bandwidth management. During high user traffic, inefficient intermediate code generation caused buffering delays and reduced streaming quality. The company plans to redesign the compiler to improve scalability and performance.

Tasks:

(i) Design intermediate code for bandwidth allocation and quality adaptation logic. *(K3)*

(ii) Explain how case statements help manage multiple streaming quality levels. *(K4)*

(iii) Analyze the significance of compiler optimization in cloud-based multimedia systems. *(K5)*

ANSWERS:

1. INDUSTRIAL ROBOT AUTOMATION SYSTEM

Problem Understanding

The manufacturing company uses robotic arms for automated assembly. The embedded compiler processes:

- Declarations
- Assignment statements
- Boolean expressions
- Procedure calls
- Safety conditions
- Robotic movement commands

The main problem is inefficient intermediate-code generation, causing delays and poor synchronization between robots. Therefore, the compiler should generate efficient intermediate representation and optimize control flow and procedure calls.

1(i) Design intermediate code for robotic movement and safety validation logic. (K3)

Scenario

Assume the robotic system has:

- x, y, z → robot coordinates
- speed → movement speed
- obstacle → obstacle detection status
- emergency → emergency-stop status
- grip → robotic gripper status

The robot should move only when: obstacle == false AND emergency == false

Otherwise, it must stop immediately.

Three-Address Code (TAC)

Suppose the high-level logic is:

```
if (obstacle == 0 && emergency == 0)
{
   move(x, y, z, speed);
   grip = 1;
}
else
{
   stop_robot();
}
```

The intermediate code can be:

```
1.  t1 = obstacle == 0
2.  ifFalse t1 goto L1
3.  t2 = emergency == 0
4.  ifFalse t2 goto L1
5.  param x
6.  param y
7.  param z
8.  param speed
9.  call move, 4
10. grip = 1
11. goto L2
12. L1:
13. call stop_robot, 0
14. L2:
15. continue
```

Explanation

TAC	Purpose
t1 = obstacle == 0	Checks whether an obstacle is absent
ifFalse t1 goto L1	Goes to emergency stop if obstacle exists
t2 = emergency == 0	Checks emergency-stop condition
ifFalse t2 goto L1	Stops robot if emergency is active
param x etc.	Passes movement parameters
call move, 4	Calls movement procedure
grip = 1	Activates gripper
goto L2	Skips stop operation
L1	Safety failure branch
call stop_robot, 0	Immediately stops robot
L2	End of conditional execution

More detailed movement validation

A practical robot may also check whether the target coordinates are within safe limits:

```
if (x >= XMIN && x <= XMAX &&
   y >= YMIN && y <= YMAX &&
   z >= ZMIN && z <= ZMAX)
```

Intermediate representation:

t1 = x >= XMIN

ifFalse t1 goto ERROR

t2 = x <= XMAX

ifFalse t2 goto ERROR

t3 = y >= YMIN

ifFalse t3 goto ERROR

t4 = y <= YMAX

ifFalse t4 goto ERROR

t5 = z >= ZMIN

ifFalse t5 goto ERROR

t6 = z <= ZMAX

ifFalse t6 goto ERROR

goto SAFE

ERROR:

call stop_robot, 0

goto END

SAFE:

param x

param y

param z

param speed

call move, 4

END:

Result

The intermediate code separates:

1. **Safety validation**
2. **Control flow**
3. **Parameter passing**
4. **Procedure calls**
5. **Robot movement**

This makes the program easier for the compiler to optimize and allows the target machine code to execute safety-critical operations quickly.

1(ii) Explain how procedure calls improve modularity in industrial automation systems. (K4)

A **procedure call** allows frequently used operations to be written once and invoked whenever required.

For example, instead of writing robot movement instructions repeatedly:

```
move(x1, y1, z1, speed1)
move(x2, y2, z2, speed2)
move(x3, y3, z3, speed3)
```

the compiler can represent the operation using a procedure:

```
procedure moveRobot(x, y, z, speed)
{
   validate_position(x, y, z);
   check_safety();
   control_motor(x, y, z, speed);
}
```

Then:

```
call moveRobot
```

How procedure calls improve modularity

1. Code Reusability

Common operations such as:

```
check_safety()
move_robot()
activate_gripper()
release_gripper()
emergency_stop()
```

can be reused by many robotic programs.

This avoids duplication.

---

2. Easier Maintenance

Suppose the safety algorithm changes.

Without procedures, every robot movement section has to be modified.

With procedures:

```
check_safety()
```

needs to be modified only once.

All calls automatically use the updated implementation.

---

3. Separation of Functions

Different procedures can represent different industrial operations:

initialize_robot()

check_safety()

move_robot()

pick_component()

place_component()

shutdown_robot()

Each procedure has a specific responsibility.

This creates a modular compiler-generated program.

---

4. Better Error Handling

Safety procedures can be centralized:

```
if (temperature > MAX_TEMP)
    emergency_stop();
```

Instead of implementing emergency handling in every movement operation, the compiler-generated code can call one common procedure.

---

5. Multiple Robot Coordination

Consider three robotic arms:

Robot1.move()

Robot2.move()

Robot3.move()

Common synchronization procedures can be used:

wait_for_robot()

lock_shared_area()

release_shared_area()

This improves coordination.

Procedure-call mechanism in intermediate code

A procedure call may be represented as:

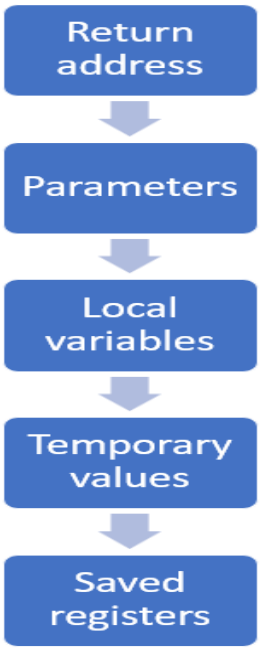
```
param x
param y
param z
param speed
call moveRobot, 4
```

Inside the compiler, procedure-call implementation involves:

1. Evaluating arguments
2. Passing parameters
3. Saving required registers
4. Creating an activation record
5. Transferring control
6. Executing procedure
7. Returning result/control

Activation Record

A procedure call may require an activation record containing:



Industrial advantage

Procedure calls allow robotic software to be:

- Modular
- Reusable
- Maintainable
- Easier to test
- Easier to synchronize
- Easier to optimize

Therefore, procedure calls are highly useful in large industrial automation systems.

1(iii) Analyze the role of compiler optimization in improving manufacturing efficiency. (K5)

Compiler optimization transforms intermediate code into an equivalent but more efficient form.

For industrial robots, optimization is important because robotic operations are often:

- Repetitive
- Time-sensitive
- Resource-constrained
- Safety-critical

Important optimizations

1. Constant Folding

Original:

```
speed = 100 * 2;
```

Optimized:

```
speed = 200;
```

The calculation is performed during compilation instead of runtime.

---

2. Constant Propagation

Original:

```
MAX_SPEED = 100;
x = MAX_SPEED;
y = x + 10;
```

Optimized:

```
x = 100;
y = 110;
```

---

3. Common Subexpression Elimination

Original:

```
t1 = x * y;
t2 = x * y + z;
```

Optimized:

```
t1 = x * y;
t2 = t1 + z;
```

The multiplication is performed only once.

---

4. Dead Code Elimination

If:

speed = 50;

speed = 100;

and the first value is never used, the compiler can remove:

speed = 50;

---

5. Strength Reduction

An expensive operation:

x = i * 2;

may be replaced in suitable low-level contexts with a cheaper equivalent operation.

---

6. Procedure Inlining

If a very small procedure is repeatedly called:

check_sensor();

the compiler may replace the call with the procedure body when appropriate.

This reduces:

- Call overhead
- Return overhead
- Parameter-passing overhead

---

7. Control-Flow Optimization

Unnecessary jumps can be eliminated.

Original:

goto L1

L1:

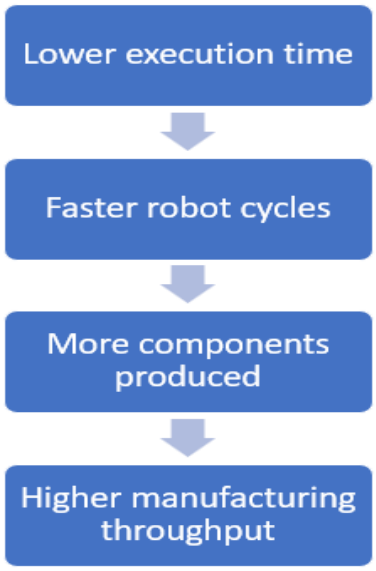
goto L2

Optimized:

goto L2

Effect on manufacturing

Optimization can result in:



It can also reduce CPU and memory consumption in embedded controllers.

Important consideration

For industrial robotics, optimization must **never change safety behavior**.

For example:

```
if (emergency)
    stop_robot();
```

must not be optimized away or reordered in a way that delays the emergency response.

Conclusion

Compiler optimization improves:

- Robot response time
- Assembly cycle time
- Controller efficiency
- Energy usage
- Memory utilization
- Synchronization
- Production throughput

Thus, compiler optimization directly contributes to manufacturing efficiency while safety-critical operations must always be preserved.

---

2. CYBERSECURITY INTRUSION DETECTION SYSTEM

2(i) Generate intermediate representation for network threat detection logic. (K3)

Scenario

Suppose the intrusion detection system checks:

```
if (failed_login > 5 && unusual_port == 1)
   alert("Possible attack");
else if (malware_signature == 1)
   block_connection();
else
   allow_connection();
```

Three-Address Code

1. t1 = failed_login > 5
2. ifFalse t1 goto L1
3. t2 = unusual_port == 1
4. ifFalse t2 goto L1
5. param "Possible attack"
6. call alert, 1
7. goto L3
8. L1:
9. t3 = malware_signature == 1
10. ifFalse t3 goto L2
11. call block_connection, 0
12. goto L3
13. L2:
14. call allow_connection, 0
15. L3:
16. continue

Explanation

The compiler converts the high-level Boolean expressions and conditional statements into:

- Temporary variables
- Conditional jumps
- Labels
- Procedure calls

This intermediate representation can then be optimized before machine-code generation.

Threat classification example

Suppose the IDS classifies traffic into:

```
if (port_scan)
   type = SCANNING;
else if (brute_force)
   type = BRUTE_FORCE;
else if (malware)
   type = MALWARE;
else
   type = NORMAL;
```

TAC:

```
if port_scan goto L1
goto L2

L1:
type = SCANNING
goto END
L2:
if brute_force goto L3
goto L4
L3:
type = BRUTE_FORCE
goto END
L4:
if malware goto L5
goto L6
L5:
type = MALWARE
goto END
L6:
type = NORMAL
END:
```

This representation clearly models the control flow required for threat detection.

2(ii) Discuss how case statements can efficiently classify multiple cyberattack scenarios. (K4)

A case statement is useful when an IDS needs to classify a large number of discrete attack types.

Suppose:

```
switch (attack_code)
{
   case 1:
      type = "Port Scan";
      break;
   case 2:
      type = "Brute Force";
      break;
   case 3:
      type = "Malware";
      break;
   case 4:
      type = "DDoS";
      break;
   default:
      type = "Unknown";
}
```

Intermediate representation

Conceptually:

```
if attack_code == 1 goto L1
if attack_code == 2 goto L2
if attack_code == 3 goto L3
if attack_code == 4 goto L4
goto L5
L1:
type = PORT_SCAN
goto END
L2:
type = BRUTE_FORCE
goto END
```

L3:

type = MALWARE

goto END

L4:

type = DDOS

goto END

L5:

type = UNKNOWN

END:

Why case statements are efficient

1. Clear Classification

Different attack categories can be represented separately:

1 → Port Scan

2 → Brute Force

3 → Malware

4 → DDoS

---

2. Jump Table Optimization

If case values are dense:

1, 2, 3, 4, 5, 6

the compiler can generate a **jump table**.

Conceptually:

target = jump_table[attack_code]

goto target

This can be faster than checking every condition sequentially.

---

3. Binary Search

If case values are sparse: 10, 50, 100, 500, 1000

the compiler may generate a decision tree or binary-search-like structure.

---

4. Easy Extension

New attack categories can be added:

case 5:

    type = SQL_INJECTION;

without redesigning the entire classification system.

---

5. Faster Real-Time Classification

Intrusion detection must process packets rapidly.

Efficient case translation reduces:

- Branching overhead
- Classification time
- CPU usage

This helps the IDS identify threats quickly.

Conclusion

Case statements are particularly useful when an intrusion detection system needs to classify many predefined attack categories. The compiler can translate them into efficient jump tables or decision trees, improving real-time threat classification.

2(iii) Evaluate the importance of compiler optimization in real-time cybersecurity applications. (K5)

Cybersecurity systems frequently have strict response-time requirements.

An IDS may process thousands or millions of network events.

If processing is slow:

Network packet

↓

Delayed analysis

↓

Delayed detection

↓

Delayed response

↓

Possible security breach

Compiler optimization reduces this delay.

Important optimizations

Constant Folding

threshold = 100 + 50;

becomes:

threshold = 150;

Dead Code Elimination

Unused detection logic can be removed.

Common Subexpression Elimination

Original:

t1 = packet_length > 1000

t2 = packet_length > 1000 && suspicious_flag

Optimized:

t1 = packet_length > 1000

t2 = t1 && suspicious_flag

Branch Optimization

Frequent attack conditions can be arranged efficiently.

Function Inlining

Small frequently used functions such as:

check_signature()

may be inlined when beneficial.

Security-specific benefit

Optimization can improve:

Area	Benefit
Packet processing	Faster analysis
Threat detection	Lower latency
CPU utilization	Reduced overhead
Memory	Lower consumption
Response time	Faster alerts
Throughput	More traffic processed

Critical limitation

Security correctness must be preserved.

For example, the compiler cannot eliminate:

```
if (malicious)
   block();
```

because it appears infrequently.

An optimization that improves speed but causes missed attacks is unacceptable.

Conclusion: Compiler optimization is essential for real-time cybersecurity because it reduces detection latency and allows more network traffic to be processed. However, optimization must preserve the correctness and security semantics of detection rules.

3. DIGITAL PAYMENT GATEWAY SYSTEM

3(i) Construct intermediate code for payment verification and fraud detection logic. (K3)

Scenario

Suppose a payment gateway performs:

```
if (amount <= balance && otp_valid)
{
   if (fraud_score < threshold)
      approve();
   else
      reject();
}
else
{
   reject();
}
```

Three-Address Code

```
1.  t1 = amount <= balance
2.  ifFalse t1 goto L1
3.  t2 = otp_valid == 1
4.  ifFalse t2 goto L1
5.  t3 = fraud_score < threshold
6.  ifFalse t3 goto L2
7.  call approve, 0
8.  goto L3
9.  L2:
10. call reject, 0
11. goto L3
12. L1:
13. call reject, 0
14. L3:
15. continue
```

Adding transaction recording

A practical payment system can additionally perform:

record_transaction()

TAC:

param transaction_id

param amount

call record_transaction, 2

Payment verification flow

Payment Request

↓

Check Balance

↓

Check OTP

↓

Calculate Fraud Score

↓

Fraud Safe?

↓

Yes | No

↓

Approve Reject

The intermediate representation makes these decisions explicit through labels and conditional jumps.

3(ii) Explain how backpatching supports conditional transaction processing. (K4)

Definition

Backpatching is a compiler technique used to fill in the target addresses of jumps when those addresses are not yet known during intermediate-code generation.

It is especially useful for:

- Boolean expressions
- if
- if-else
- while
- switch
- Conditional transaction processing

Example

Consider:

```
if (balance >= amount && otp_valid)
   approve();
else
   reject();
```

While generating code, the compiler may not yet know the exact instruction numbers of:

approve

reject

So it generates incomplete jumps.

For example:

1. if balance >= amount goto _
2. goto _
3. if otp_valid goto _
4. goto _
5. approve()
6. goto _
7. reject()

The _ values represent unknown target addresses.

Backpatching fills them later.

Backpatching lists

For Boolean expressions, the compiler maintains lists such as:

- truelist
- falselist
- nextlist

For:

balance >= amount && otp_valid

The compiler creates:

balance >= amount

with a true and false destination.

Because this is an AND expression:

A && B

the true branch of A must lead to evaluation of B.

Conceptually:

A true

↓

evaluate B

↓

B true → approve

B false → reject

A false → reject

Example of completed code

After backpatching:

1. if balance >= amount goto 3
2. goto 7
3. if otp_valid goto 5
4. goto 7
5. call approve, 0
6. goto 8
7. call reject, 0
8. continue

Why it is useful in payment systems

Backpatching allows the compiler to construct control flow without knowing all target addresses initially.

It supports:

- Conditional authorization
- Fraud detection
- Rollback decisions
- Payment validation
- Multi-stage verification

Example with rollback

```
if payment_verified
   commit_transaction();
else
   rollback_transaction();
```

Intermediate code:

```
if payment_verified goto L1
goto L2
L1:
```

call commit_transaction, 0

goto L3

L2:

call rollback_transaction, 0

L3:

Backpatching determines the correct labels for these branches during intermediate-code generation.

3(iii) Analyze the role of compiler design in ensuring secure and scalable fintech applications. (K5)

Compiler design has an important role in payment processing because fintech applications require:

- Security
- Speed
- Reliability
- Scalability
- Correctness

1. Efficient Intermediate Representation

A good IR provides an effective bridge between source code and machine code.

For example:

Source Program

↓

Intermediate Representation

↓

Optimization

↓

Machine Code

Efficient IR allows payment validation logic to be optimized.

---

2. Security

Compiler design can help preserve security-sensitive operations such as:

verify_otp()

check_balance()

detect_fraud()

authorize()

rollback()

The compiler must preserve the required ordering and semantics.

---

3. Optimization

Suppose a fraud calculation contains:

t1 = amount * risk_factor

t2 = amount * risk_factor + previous_score

The compiler can use:

t1 = amount * risk_factor

t2 = t1 + previous_score

This reduces repeated computation.

---

4. Scalability

During peak periods, millions of transactions may be processed.

Efficient generated code allows more transactions to be handled using the same computing resources.

---

5. Reliable Control Flow

Payment processing often contains:

```
if verified
   approve
else
   reject
```

and:

```
if failed
   rollback
```

Correct control-flow generation is essential because incorrect branching could lead to:

- Incorrect approval
- Incorrect rejection
- Transaction inconsistency
- Financial loss

---

6. Procedure Calls

Functions such as:

validate_payment()

detect_fraud()

authorize_transaction()

rollback_transaction()

provide modularity and allow independent components to be maintained and tested.

---

7. Runtime Performance

Compiler optimization can reduce:

- CPU usage
- Memory usage
- Execution time

This improves transaction throughput.

Conclusion

Compiler design provides the foundation for generating efficient and reliable payment-processing software. Optimization improves scalability, while correct control-flow and procedure handling preserve transaction security and reliability.

---

4. SMART AGRICULTURE MONITORING SYSTEM

4(i) Develop intermediate code for irrigation control and environmental monitoring. (K3)

Scenario

Assume the agricultural system monitors:

- moisture
- temperature
- rain
- water_level

Irrigation should start when:

moisture < 30

AND

rain == 0

AND

water_level > 20

Three-Address Code

1. t1 = moisture < 30
2. ifFalse t1 goto L1
3. t2 = rain == 0
4. ifFalse t2 goto L1
5. t3 = water_level > 20
6. ifFalse t3 goto L1

7. call start_irrigation, 0
8. goto L2
9. L1:
10. call stop_irrigation, 0
11. L2:
12. continue

Temperature monitoring

Suppose:

```
if temperature > 40
   alert_high_temperature();
```

TAC:

```
t4 = temperature > 40
ifFalse t4 goto L3
call alert_high_temperature, 0
L3:
```

Complete environmental control

```
t1 = moisture < 30
ifFalse t1 goto L
t2 = rain == 0
ifFalse t2 goto L1
t3 = water_level > 20
ifFalse t3 goto L1
call start_irrigation, 0
goto L2
L1:
call stop_irrigation, 0
L2:
t4 = temperature > 40
ifFalse t4 goto L3
call alert_high_temperature, 0
L3:
continue
```

This IR clearly represents the automated decision-making process.

4(ii) Explain the role of Boolean expressions in automated farming systems. (K4)

Boolean expressions are essential because smart agriculture systems make decisions based on sensor values.

For example:

moisture < 30

is a Boolean expression that evaluates to:

TRUE or FALSE

---

1. Irrigation Control

moisture < 30 && rain == 0

means:

- Soil is dry
- It is not raining

Therefore:

Start irrigation

---

2. Multiple Sensor Conditions

A more complete condition can be:

moisture < 30 &&

temperature > 20 &&

rain == 0 &&

water_level > 20

The irrigation system operates only when all required conditions are satisfied.

---

3. OR Conditions

Consider:

temperature > 40 || humidity < 20

This means an alert should be generated if either:

- Temperature is too high, or
- Humidity is too low.

---

4. NOT Operator

For example:

!rain_detected

means:

rain_detected == false

It can be used to prevent unnecessary irrigation.

Boolean expression in intermediate code

For:

moisture < 30 && rain == 0

the compiler may generate:

t1 = moisture < 30

ifFalse t1 goto L1

t2 = rain == 0

ifFalse t2 goto L1

call start_irrigation, 0

This shows how Boolean expressions become conditional branches in intermediate code.

Importance

Boolean expressions provide:

- Automatic decision making
- Sensor-based control
- Reduced human intervention
- Real-time response
- Precise environmental management

Thus, Boolean expressions form the decision-making foundation of smart farming systems.

4(iii) Evaluate how compiler optimization enhances efficiency in smart agriculture applications. (K5)

Smart agriculture controllers often operate on embedded devices with limited:

- CPU
- Memory
- Battery
- Processing power

Therefore, optimized code is important.

1. Constant Folding

irrigation_time = 10 * 60;

can become:

irrigation_time = 600;

---

2. Dead Code Elimination

If a sensor value is read but never used:

humidity = read_humidity();

and humidity is never referenced, unnecessary operations can potentially be eliminated.

---

3. Common Subexpression Elimination

Original:

t1 = temperature > 40;

t2 = temperature > 40 && moisture < 20;

Optimized:

t1 = temperature > 40;

t2 = t1 && moisture < 20;

---

4. Loop Optimization

Agricultural systems may continuously monitor sensors:

```
while (true)
{
   read_sensor();
   process_data();
}
```

The compiler can optimize loop operations to reduce unnecessary computations.

---

5. Reduced Energy Consumption

Faster execution means the processor may spend less time actively executing code.

This is important for:

- Solar-powered sensors
- Battery-operated IoT devices
- Remote agricultural monitoring systems

---

6. Faster Real-Time Response

Suppose: moisture < critical_level

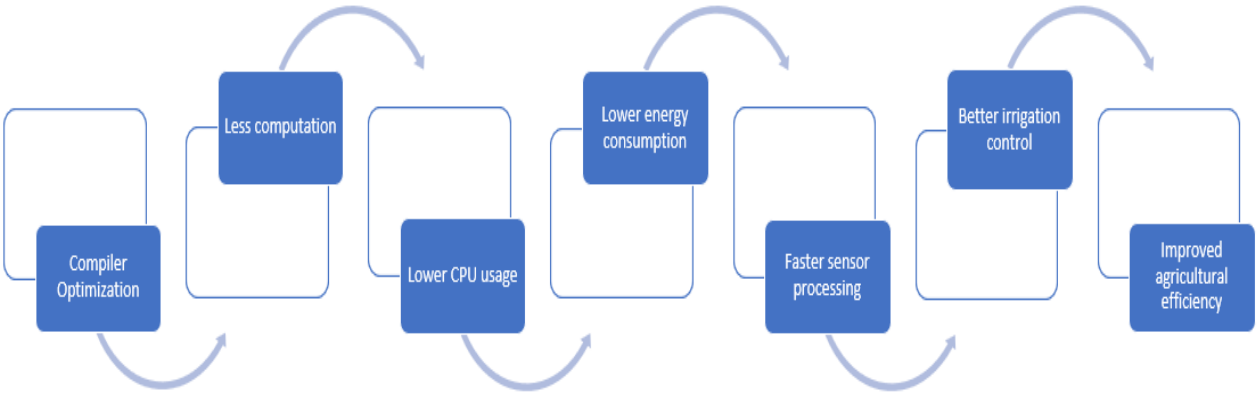
is detected.

The optimized controller can quickly execute:

start_irrigation();

This reduces the delay between sensor detection and physical action.

Overall impact



Conclusion

Compiler optimization makes smart agriculture systems faster, more energy-efficient and more responsive. It is especially valuable for embedded controllers and real-time sensor-based automation.

---

5. CLOUD-BASED VIDEO STREAMING PLATFORM

5(i) Design intermediate code for bandwidth allocation and quality adaptation logic. (K3)

Scenario

Suppose the streaming system selects video quality based on available bandwidth.

Rules:

```
if bandwidth >= 10 Mbps
    quality = HIGH;
else if bandwidth >= 5 Mbps
    quality = MEDIUM;
else
    quality = LOW;
```

Three-Address Code

1. t1 = bandwidth >= 10
2. ifFalse t1 goto L1
3. quality = HIGH
4. goto L3
5. L1:
6. t2 = bandwidth >= 5
7. ifFalse t2 goto L2

8. quality = MEDIUM
9. goto L3
10. L2:
11. quality = LOW
12. L3:
13. continue

Adding buffer status

Suppose quality should also decrease when buffer level is low:

```
if buffer < 10
   quality = LOW;
```

TAC:

```
t3 = buffer < 10
ifFalse t3 goto L4
quality = LOW
L4:
continue
```

Bandwidth allocation

Suppose:

```
available_bandwidth = total_bandwidth - reserved_bandwidth;
```

TAC:

```
t1 = total_bandwidth - reserved_bandwidth
available_bandwidth = t1
```

Then:

```
if available_bandwidth >= 10
   quality = HIGH
else if available_bandwidth >= 5
   quality = MEDIUM
else
   quality = LOW
```

The compiler converts these conditions into labels, jumps and temporary variables.

5(ii) Explain how case statements help manage multiple streaming quality levels. (K4)

A streaming platform may support:

1 → 144p

2 → 360p

3 → 480p

4 → 720p

5 → 1080p

6 → 4K

A case statement can efficiently select the quality:

```
switch (quality_code)
{
   case 1:
      resolution = 144;
      break;
   case 2:
      resolution = 360;
      break;
   case 3:
      resolution = 480;
      break;
   case 4:
      resolution = 720;
      break;
   case 5:
      resolution = 1080;
      break;
   case 6:
      resolution = 2160;
      break;
   default:
      resolution = 360;
}
```

Intermediate representation

Conceptually:

```
if quality_code == 1 goto L1
if quality_code == 2 goto L2
```

if quality_code == 3 goto L3

if quality_code == 4 goto L4

if quality_code == 5 goto L5

if quality_code == 6 goto L6

goto DEFAULT

L1:

resolution = 144

goto END

L2:

resolution = 360

goto END

L3:

resolution = 480

goto END

L4:

resolution = 720

goto END

L5:

resolution = 1080

goto END

L6:

resolution = 2160

goto END

DEFAULT:

resolution = 360

END:

Jump table

Since quality levels are consecutive values, the compiler can potentially use a jump table:

jump_table[quality_code]

For example:

quality_code = 4

↓

jump_table[4]

↓

720p

This avoids checking every case one by one.

Benefits for video streaming

Faster Quality Selection

The system can quickly change:

4K → 1080p → 720p → 480p

depending on network conditions.

Better Adaptation

During network congestion:

Bandwidth decreases

↓

Quality code changes

↓

Case statement selects lower quality

↓

Streaming continues

Conclusion

Case statements provide a structured and efficient way to manage multiple video-quality levels, while compiler techniques such as jump tables can make quality selection faster.

5(iii) Analyze the significance of compiler optimization in cloud-based multimedia systems. (K5)

Cloud-based streaming systems must handle huge numbers of:

- Users
- Video streams
- Network requests
- Quality adaptations
- Resource-allocation operations

Therefore, compiler optimization is important for both performance and scalability.

1. Faster Quality Adaptation

The streaming engine continuously evaluates:

bandwidth

buffer_level

network_latency

packet_loss

Efficiently compiled conditions allow faster quality changes.

2. Reduced CPU Usage

Suppose the same calculation appears repeatedly:

available = bandwidth - reserved;

The compiler can avoid recalculating it unnecessarily.

This reduces server CPU usage.

3. Common Subexpression Elimination

Original:

t1 = bandwidth - reserved

t2 = bandwidth - reserved + buffer

Optimized:

t1 = bandwidth - reserved

t2 = t1 + buffer

Only one subtraction is required.

4. Dead Code Elimination

Unused streaming calculations can be removed.

This reduces unnecessary instructions and improves execution speed.

5. Function Inlining

Frequently called small functions such as:

check_bandwidth()

or:

select_quality()

may be inlined when beneficial.

This can reduce function-call overhead.

6. Loop Optimization

Streaming systems frequently perform operations repeatedly:

```
while (stream_active)
{
   measure_bandwidth();
   update_buffer();
   select_quality();
}
```

Optimizing this loop can significantly improve performance because it executes continuously.

7. Better Cloud Scalability

Suppose an unoptimized server handles:

10,000 streams

and optimization reduces CPU usage per stream.

The same hardware may then support more streams.

Therefore:

Compiler Optimization

↓

Lower CPU cost per stream

↓

More streams per server

↓

Better resource utilization

↓

Improved scalability

8. Important Optimization Techniques

Optimization	**Streaming Benefit**
Constant folding	Removes compile-time calculations
Constant propagation	Reduces repeated value lookup
Common subexpression elimination	Avoids repeated calculations
Dead-code elimination	Removes unnecessary instructions
Loop optimization	Improves repeated processing
Function inlining	Reduces call overhead
Control-flow optimization	Reduces unnecessary branches

Overall Evaluation

The streaming pipeline can be represented as:

Network Conditions

↓

Bandwidth Monitoring

↓

Quality Decision

↓

Optimized Intermediate Code

↓

Fast Execution

↓

Video Quality Adaptation

↓

Smooth Streaming

Conclusion

Compiler optimization is highly significant in cloud-based multimedia systems because it improves execution speed, reduces CPU and memory consumption, minimizes buffering, and allows cloud infrastructure to serve more users with the same resources. The compiler therefore contributes directly to both **streaming performance and cloud scalability**.
